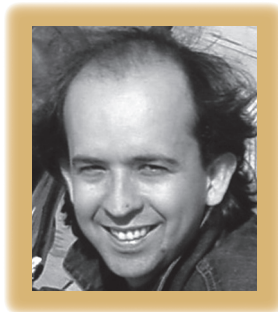


Five Considerations for Software Architecture, Part I

Frank Buschmann and Kevlin Henney

What are the top five properties that make a software design both effective and elegant? That's a really challenging question, isn't it? Our colleague Charles Weir asked it at the 2001 ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA) in Tampa, Florida, and discussed it at



the Four Green Fields Irish pub at the end of a long conference day. It took us a while to get a handle on the topic, but a few pints of Guinness really helped us gain the necessary focus.

In the end we came up with the five properties—or better yet, five considerations. Considerations is a better term because the properties we found are weaker than the general notion of rules and recommendations. Rather, they provide a perspective on software architecture, a value system that can broadly guide architects' design decisions. The five considerations are economy, visibility, spacing, symmetry, and emergence.

This column and the next one explore these five simple words.

Economy

Many software architectures suffer from unnecessary, accidental complexity: arbitrary flexibility for its own sake, unnecessary features, design choices whose complexity is out of proportion for the problems and requirements at hand, or a focus on reusability rather than usability.¹⁻³

Figure 1 shows a “reusable” interface from the mostly unused Corba as per “OptionalStyles.dt” style guide collections service.

Leaving aside whether it makes sense to define a general-purpose traversal interface for arbitrary collection objects on a network, this interface makes an impressive meal of the apparently simple task of iterating over a collection.

What we really want is something like Figure 2. This interface is much simpler. It has only a few methods, whose purpose is clear and whose use specifically suits a networked environment. Overall, *BindingIterator* follows the Iterator pattern. The method *next_one()* follows the combined method pattern, and the method *next_n()* follows the Batch Method pattern.⁴

Just considering simplicity or the sheer reduction of complexity isn't sufficient. A simple design isn't necessarily usable—in such cases, developers will find reasons to not use it, work around it, or break the design in its implementation. An economic design, by contrast, balances clarity, compression, and abstraction. Clarity is achieved by reducing clutter, compression relates to the directness of the design, and abstraction concerns the removal of unwanted detail.

From a methodological perspective, designing

```

interface Iterator
{
    boolean set_to_first_element();
    boolean set_to_next_element();
    boolean set_to_next_nth_element(in unsigned long n) raises(...);
    boolean retrieve_element(out any element) raises(...);
    boolean retrieve_element_set_to_next(out any element, out boolean more) raises(...);
    boolean retrieve_next_n_elements
        (in unsigned long n, out AnySequence result, out boolean more) raises(...);
    boolean not_equal_retrieve_element_set_to_next(in Iterator test, out any element) raises(...);
    void remove_element() raises(...);
    boolean remove_element_set_to_next() raises(...);
    boolean remove_next_n_elements(in unsigned long n, out unsigned long actual_number) raises(...);
    boolean not_equal_remove_element_set_to_next(in Iterator test) raises(...);
    void replace_element(in any element) raises(...);
    boolean replace_element_set_to_next(in any element) raises(...);
    boolean replace_next_n_elements
        (in AnySequence elements, out unsigned long actual_number) raises(...);
    boolean not_equal_replace_element_set_to_next(in Iterator test, in any element) raises(...);
    boolean add_element_set_iterator(in any element) raises(...);
    boolean add_n_elements_set_iterator
        (in AnySequence elements, out unsigned long actual_number) raises(...);
    void invalidate();
    boolean is_valid();
    boolean is_in_between();
    boolean is_for(in Collection collector);
    boolean is_const();
    boolean is_equal(in Iterator test) raises(...);
    Iterator clone();
    void assign(in Iterator from_where) raises(...);
    void destroy();
};

```

```

interface BindingIterator
{
    boolean next_one(out Binding result);
    boolean next_n(in unsigned long how_many, out BindingList result);
    void destroy();
};

```

Figure 1. Sketch of the Iterator interface from the Corba collections service. Many complex methods that aren't meaningful in the scope of traversing the elements of a remote collection make this interface overly complex, hard to understand, and difficult to use.

Invisibility arises mainly when important design and domain concepts are missing or are at best implied rather than visible in the system's code. For example, few variables of type `string` are free of constraints and interpretation. Often, they represent an implied and associated concept—for instance, an ISBN (International Standard Book Number) or an SQL (Structured Query Language) statement. Although ISBNs and SQL statements can reasonably be transported and persisted via strings, this isn't the most effective in-program representation.

Visibility means making the implicit explicit. In our example, a type that expresses the domain type directly and forces it to be well formed makes the concept explicit. The internal use of any `string` is encapsulated. Introducing an explicit type for a concept also reduces duplicated code and unspoken design assumptions, increasing opportunities for optimization and representation changes. Other approaches that improve visibility include grouping items that are passed around together and treated as a whole, and providing names for components and their relationships that clearly indicate their purpose and responsibility.

Domain-driven design (DDD) is rooted in the idea of visibility.⁷ One of its key messages is the direct, explicit portrayal of a system's domain model in its architecture. It doesn't matter whether the domain is an application domain, such as medical imaging, or a technical domain, such as persistence. What matters is the concepts' explicit visibility.

Yet visibility also has an overdose effect. If everything is fully visible, you might not see the forest for the trees. Concepts that are essential for an architecture's success should be visible and tangible. Less important concepts might be less visible, which in turn emphasizes the important concepts' visibility. A previous column described a war story from the design of an industrial

Figure 2. A simple interface for iterating over remote collections. This interface is lean and economical because it focuses on the minimal set of methods useful for traversing the elements of a remote collection.

toward requirements and responsibilities is a good approach to balance these three factors. The XDD family of approaches—in particular, test-driven, responsibility-driven, and domain-driven design and development^{5–7}—set the architect's focus on what to design, identifying usable architecture concepts more naturally. The design-tactics approach outlined in the March/April edition of this column addresses simplicity by identifying the simplest solutions for realizing the concepts to be expressed.²

Visibility

Visibility relates to the directness and expressiveness of the artifacts that describe and constitute software architectures. Too often, interfaces have poor expressiveness, and the significance of their partitioning is obscure. Documentation is either completely lacking or excessive and poorly communicated. Obvious consequences include poor modifiability and maintainability, along with steep learning curves for newcomers.

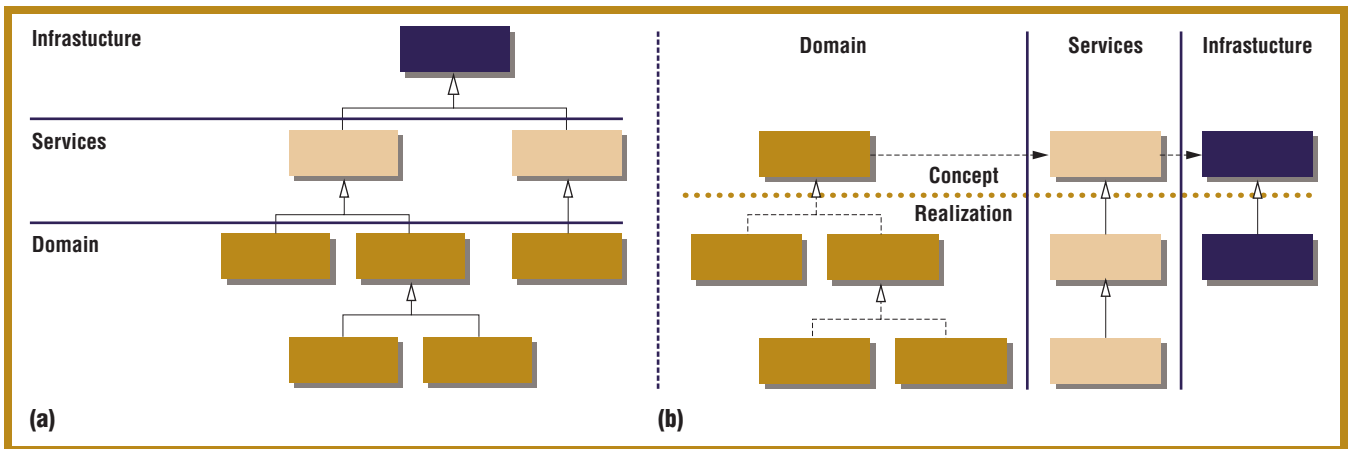


Figure 3. Two options for separation of concerns. (a) This option uses multiple hierarchy levels, which can lead to overloaded leaf classes. (b) This option concerns hierarchy separation and composition, which support modular designs with clearly visible and distinguishable concepts.

object model, in which its architects modeled not only the different types of industrial objects as explicit classes, but also all object attributes.² DDD guided many of the refactoring activities, which significantly improved the visibility of the key abstractions in the object model's design.

Patterns are also a means for making concepts visible. For instance, Explicit Interfaces reify the roles components offer to their clients, Strategy reifies the algorithms components use, Value Objects reify the simple pieces of information in a domain, and Layers reify a software system's abstraction levels.³ Looked at through "visibility glasses," many patterns support making architectural concepts more visible.

Spacing

Spacing occurs in many aspects of software and its development, whether in the literal interpretation of spacing in source code or the more conceptual spacing of separating concerns in a modular design. Spacing can enhance visibility; it distinguishes each concept more clearly from others.

Yet, useful spacing isn't simply achieved by any form of separation. For example, the class hierarchy in Figure 3a is a common approach for organizing hierarchies: infrastructural plumbing at the root, additional adaptation beneath that, and actual domain classes at the leaves. It exhibits layering of concepts, and therefore some spacing between them. However, this use of inheritance to accumulate through layers steadily increases the dependencies that

any leaf class must contend with. A degree of hierarchy lock-in and overhead arises from insufficient separation between the conceptual layers.

Spacing through hierarchy separation and composition offers an alternative with greater composability and orthogonality (see Figure 3b). The focus is more naturally on the domain first and infrastructure as a tertiary detail, and not necessarily one that must intrude much beyond the services layer. Delegation through conceptual roots expressed as pure interfaces leads to a stricter, more separated layering.

Sufficient spacing naturally leads to modular designs with clearly visible and distinguishable concepts:

- Spacing between clearly distinct and self-contained functional responsibilities leads to components.
- Spacing between a component's usage perspectives leads to role-specific interfaces.
- Spacing between groups of components leads to layering and subsystems.
- Spacing between contract and realization leads to explicit interfaces and separated implementations.
- Spacing between commonalities and variabilities leads to stable framework architectures with defined plug-in concepts for variabilities

Yet all spacing must be balanced. Too much spacing leads to fragmentation and loss of visibility. Methods such as DDD and commonality and variability analysis help identify what to separate in a de-

sign and what to treat as a whole, coherent unit.⁸

The next installment of this column will explore symmetry and emergence and discuss how these considerations trade off of one another and keep each other in check. ☞

References

1. F. Buschmann, "Learning from Failure, Part 2: Performatitis, and Other Diseases," *IEEE Software*, vol. 27, no. 1, 2010, pp. 10–11.
2. F. Buschmann, "Learning from Failure, Part 3: On Hammers and Nails, and Falling in Love with Technology and Design," *IEEE Software*, vol. 27, no. 2, 2010, pp. 49–51.
3. F. Buschmann, K. Henney, and D.C. Schmidt, *Pattern-Oriented Software Architecture—A Pattern Language for Distributed Computing*, vol. 4, John Wiley & Sons, 2007.
4. K. Henney, "Simplicity before Generality, Use before Reuse," *97 Things Every Software Architect Should Know*, Richard Monson-Haefel, ed., O'Reilly, 2009, pp. 36–37.
5. K. Beck, *Test-Driven Development: By Example*, Addison-Wesley Professional, 2002.
6. R. Wirfs-Brock and A. McKean, *Object Design: Roles, Responsibilities, and Collaborations*, Addison-Wesley, 2002.
7. E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2004.
8. D.M. Weiss and C.T.R. Lai, *Software Product-Line Engineering: A Family-Based Software Development Process*, Addison-Wesley, 1999.

Frank Buschmann is a principal engineer at Siemens Corporate Technology, where he's a principal software architect in the Software and Engineering Division's Architecture Department. Contact him at frank.buschmann@siemens.com.

Kevlin Henney is an independent consultant dealing with software architecture, development processes, and programming techniques and practices. Henney is a member of the *IEEE Software Advisory Board*. Contact him at kevin@curbralan.com.